Final Exam Master Review

Operations Research — Comprehensive Summary

Table of Contents

---

1. Game Theory

Core Concept

A mathematical framework for **competitive decision-making** between rational players. This course focuses on **two-person, zero-sum games**: one player's gain = the other's loss.

Key Vocabulary

Term	Meaning
Strategy	A complete plan of action for a player
Payoff table	Matrix of outcomes (Player 1's perspective)
Saddle point	A cell that is the row minimum AND column maximum simultaneously — the stable equilibrium
Value of the game	The payoff at the saddle point (or expected payoff with optimal mixed strategies)
Zero-sum	Total payoff = 0 across both players at all times

Method 1: Dominated Strategies

> Use when one strategy is always worse than another regardless of opponent's move.

Steps: 1. For **Player 1** (maximizer): Drop any row where another row is always in all columns. 2. For **Player 2** (minimizer): Drop any column where another column is always in all rows. 3. Repeat until no more strategies can be eliminated. 4. The remaining cell is the **pure strategy solution**.

Example (Variation 1): - Player 1's Strategy 3 (0, 1, −1) is dominated by Strategy 1 (1, 2, 4) → drop it. - Player 2's Strategy 3 (4, 5) is dominated → drop it. - Continue until only Strategy 1 vs Strategy 1 remains → **value = 1,000 votes**.

Method 2: Minimax Criterion (Pure Strategy)

Use when no dominated strategies exist but a saddle point does.

Player 1 (Maximin): For each row, find the minimum. Choose the **row with the maximum of those minimums**.

Player 2 (Minimax): For each column, find the maximum. Choose the **column with the minimum of those maximums**.

Saddle Point exists if: maximin value = minimax value.

Example (Variation 2): - Row minimums: (−3, 0, −4) → maximin = **0** (Strategy 2) - Column maximums: (5, 0, 6) → minimax = **0** (Strategy 2) - Saddle point at (Strategy 2, Strategy 2), **value = 0** (fair game).

Method 3: Mixed Strategies (No Saddle Point)

Use when the game has no saddle point (instability — players keep switching pure strategies).

Definition: Each player assigns probabilities to their strategies. - Player 1: probabilities (x , x , …, x) where Σx = 1 - Player 2: probabilities (y , y , …, y) where Σy = 1

Expected Payoff:

$$E = \sum_{i,j} p_{ij} \cdot x_i \cdot y_j$$

Goal: Player 1 **maximizes** minimum expected payoff; Player 2 **minimizes** maximum expected payoff.

Minimax Theorem: There always exists a stable pair of mixed strategies where maximin = minimax = **v** (value of the game).

Method 4: Graphical Solution (One Player Has 2 Strategies)

1. Drop dominated strategies so one player has only 2 options.
2. Let x = probability of Strategy 1 for that player (x $= 1 -$ x).
3. Write expected payoff equations against each of the opponent's strategies.
4. Plot lines on a graph (x-axis: x from 0 to 1, y-axis: payoff).
5. **Player 1 (maximin):** Find the highest point of the lower envelope (intersection of the two lowest lines).
6. Solve the two intersecting lines simultaneously to find optimal x .

Variation 3 Result: - Drop Strategy 3 (dominated by Strategy 2). - Lines: y $= 5 - 5$x , y $= 4 - 6$x , y $= -3 + 5$x - Intersection of Lines 2 & 3: $-3 + 5$x $= 4 - 6$x $\rightarrow$ **x $= 7/11$, x $= 4/11$ - Value $= 2/11$ 182 votes** - Player 2 optimal: y $= 0$, y $= 5/11$, y $= 6/11$

Method 5: Linear Programming Formulation

Player 1's LP (Maximizing):

$$\text{Maximize } Z = x_{m+1} \ (= \text{v})$$

Subject to: For each of Player 2's strategies j:

$$\sum_i p_{ij} x_i \geq x_{m+1}, \quad \sum x_i = 1, \quad x_i \geq 0$$

Player 2's LP is the **dual** of Player 1's — solving one gives both answers.

Key insight: By strong duality: Max x = Min y $\rightarrow$ proves the minimax theorem.

> **If v might be negative:** Add a constant K to all payoffs, solve, then subtract K from the result. Strategies remain unchanged.

Exam Quick-Reference

Situation	Method
One strategy always worse	**Dominated Strategies**
Saddle point exists	**Minimax Criterion (Pure)**
No saddle point, one player has 2 strategies	**Graphical Method**
No saddle point, general case	**Linear Programming**

---

2. Decision Analysis & Utility Theory

The Big Picture: What is Decision Analysis?

Decision Analysis provides a **structured, quantitative framework** for choosing among alternatives under uncertainty. It answers: *"Given that the future is unknown, what is the best action to take now?"*

Core elements in every decision problem: - **Alternatives** — the choices available to the decision-maker (what we control). - **States of Nature** — uncertain future events that affect outcomes (what we don't control). - **Payoffs** — the consequence (reward or cost) for each combination of alternative and state. - **Probabilities** — likelihood assigned to each state of nature.

Payoff Tables

A payoff table organizes all combinations of alternatives and states of nature into a matrix. Each cell = payoff if that alternative is chosen AND that state occurs.

	State 1 (Oil)	State 2 (Dry)
Drill	+700	−100
Sell	+90	+90

Prior probability = initial belief about each state before any additional information. **Posterior probability** = updated belief after gathering information (via Bayes' theorem).

Decision Criteria (Pre-Utility / Without Probabilities)

These are used when probabilities are **unknown or unreliable**:

1. Maximin (Wald's criterion — pessimistic): - For each alternative, find the worst-case payoff. - Choose the alternative with the **best worst-case** payoff. - Suitable for: highly risk-averse decision-makers.

2. Maximax (optimistic): - For each alternative, find the best-case payoff. - Choose the alternative with the **best best-case** payoff. - Suitable for: risk-seeking decision-makers.

3. Minimax Regret (Savage criterion): - Compute the **regret table**: for each state, subtract each payoff from the maximum payoff in that column. - For each alternative, find its maximum regret. - Choose the alternative that **minimizes maximum regret**.

4. Equally Likely (Laplace criterion): - Assume all states of nature are equally probable. - Compute the average payoff for each alternative. - Choose the alternative with the **highest average**.

Bayes' Decision Rule (With Probabilities)

When probabilities **are** available, use **Expected Payoff (EP)**:

$$EP(\text{alternative}) = \sum_j p_j \cdot \text{payoff}_{ij}$$

Where p = prior probability of state j.

Choose the alternative with the highest EP. This is also called the **Bayes' criterion** or **expected monetary value (EMV)** criterion.

Goferbroke example: - P(oil) = 0.25, P(dry) = 0.75 - EP(Drill) = 0.25(700) + 0.75(−100) = 175 − 75 = **$100K** - EP(Sell) = 0.25(90) + 0.75(90) = **$90K** - **Optimal: Drill** (without any survey information)

Value of Information

Expected Value of Perfect Information (EVPI): > How much would we pay for a crystal ball that tells us the true state before we decide?

$$EVPI = EP(\text{with perfect info}) - EP(\text{best without info})$$

- EP with perfect info = Σ p × (best payoff in state j)
- Goferbroke: EP(PI) = 0.25(700) + 0.75(90) = 175 + 67.5 = **$242.5K**
- EVPI = 242.5 − 100 = **$142.5K** → maximum you'd pay for a perfect seismic survey.

Expected Value of Sample Information (EVSI): > Value of imperfect information (e.g., a real survey that is only sometimes correct).

$$EVSI = EP(\text{with sample info}) - EP(\text{without sample info})$$

EVSI EVPI always (imperfect info is worth less than perfect info).

Efficiency of sample information:

$$\text{Efficiency} = \frac{EVSI}{EVPI} \times 100\%$$

Bayesian Updating (Prior → Posterior)

When a test or survey provides additional information, update probabilities using **Bayes' theorem**.

Setup notation: - P(S) = prior probability of state i - P(F | S) = conditional probability of finding F given state i (from historical accuracy data) - P(S | F) = posterior probability of state i given finding F

Bayes' Theorem:

$$P(S_i|F) = \frac{P(F|S_i) \cdot P(S_i)}{\sum_j P(F|S_j) \cdot P(S_j)}$$

Goferbroke seismic survey example:

Given: P(oil)=0.25, P(dry)=0.75. Survey accuracy: P(USS | oil)=0.6, P(USS | dry)=0.2

	P(S)	P(USS | S)	Joint = P(S)×P(USS|S)	Posterior P(S |USS)
Oil	0.25	0.6	0.15	0.15/0.25 = **0.60**
Dry	0.75	0.2	0.10	0.10/0.25 = **0.40**
Total			**P(USS) = 0.25**	

So after an "unfavorable" seismic signal, P(oil | USS) rises to 0.60 — you drill with more confidence.

Decision Tree Review

A decision tree maps out all decisions and uncertain events in chronological order (left → right), then solves by **backward induction** (right → left).

Node types: - **Square** = Decision node — the decision-maker chooses; pick branch with highest EP/EU. - **Circle** = Event (chance) node — nature chooses; compute expected value as Σ(p × outcome).

Procedure: 1. Build the tree left to right (decisions first, then chance events, then payoffs at the tips). 2. Solve right to left: - At event nodes: compute EP = Σ(probability × downstream value). - At decision nodes: take the maximum EP; mark inferior branches with "//". 3. The optimal strategy is the sequence of decisions that traces the un-marked path.

Goferbroke Decision Tree EP values: - Node f (drill, unfavorable survey): EP = (1/7)(670) + (6/7)(−130) = **−$15.7K** → Sell instead - Node g (drill,

favorable survey): EP = (1/2)(670) + (1/2)(−130) = **$270K** → Drill - Node h (drill, no survey): EP = (1/4)(700) + (3/4)(−100) = **$100K**

Why Utility Theory?

Bayes' Decision Rule maximizes **expected monetary value** — but money is not always the right unit. Problems arise when: - The decision-maker has **limited capital** (a loss could be catastrophic). - **Psychological factors** make large losses feel worse than equivalent gains feel good. - The decision-maker's **risk attitude** differs from risk-neutral.

Utility theory translates money into **subjective value U(x)**, then maximizes **expected utility (EU)** instead of expected monetary payoff.

Risk Attitudes — Deep Dive

Attitude	Characteristic	Utility Curve Shape	Example Behavior
Risk-Averse	Extra cautious; prefers certainty	Concave (bows left)	Buys insurance; prefers guaranteed $90K over 50/50 chance of $0 or $200K

Attitude	Characteristic	Utility Curve Shape	Example Behavior
Risk-Neutral	Pure EMV maximizer	Linear (45° line)	Indifferent between \$100K certain and 50/50 of \$0/\$200K
Risk-Seeking	Gambler mentality	Convex (bows right)	Prefers lottery ticket over certain small gain

Decreasing marginal utility: Each additional dollar is worth less utility than the previous one. This is the hallmark of risk-aversion. Example: U(\$30K) $<$ 2 $\times$ U(\$10K) and U(\$100K) $<$ 2 $\times$ U(\$30K).

The Goferbroke owner: Naturally risk-seeking but **forced into risk-aversion** by financial constraints — a devastating −\$100K loss would be catastrophic even though −\$130K is technically the worst case. This is why utility theory changes the optimal decision.

Constructing a Utility Function

Step 1: Set arbitrary reference utilities at the extreme payoffs: - U(worst payoff) = 0 $\rightarrow$ U(−130) = 0 - U(best payoff) = 1 $\rightarrow$ U(700) = 1

Step 2: Equivalent lottery method for intermediate values. Ask the decision-maker: *"What probability p makes you indifferent between receiving \$M for certain vs. a lottery giving p chance of the best outcome and (1−p) chance of the worst outcome?"*

$$U(M) = p \cdot U(\max) + (1-p) \cdot U(\min) = p \cdot 1 + (1-p) \cdot 0 = p$$

So whatever probability they name = their utility for M. This is **revealed preference** — utility is not calculated, it is elicited from choices.

Goferbroke elicited points: | M | p (indifference probability) | U(M) | |—|—|—| | −130 | — | **0** (reference) | | −100 | 1/20 = 0.05 | **0.05** | | 90 | 1/3 0.333 | **0.333** | | 700 | — | **1** (reference) |

Step 3: Draw a smooth curve through all points.

Step 4: Read off or interpolate values not directly elicited:

$$U(x) = U(x_1) + \frac{x - x_1}{x_2 - x_1}[U(x_2) - U(x_1)]$$

Example — U(60) using bracketing points (−100, 0.05) and (90, 0.333):

$$U(60) = 0.05 + \frac{60 - (-100)}{90 - (-100)}(0.333 - 0.05) = 0.05 + \frac{160}{190}(0.283) \approx 0.288 \approx 0.30$$

Additional interpolated values: - U(60) 0.30 - U(670) 0.97

Decision Tree with Utilities

Replace all terminal monetary payoffs with their utility values. Then compute **expected utility (EU)** at each node using the same backward induction procedure.

Goferbroke utility values used: | Payoff | U(payoff) | |———|————| | 700 | 1.00 | | 670 | 0.97 | | 90 | 0.333 | | 60 | 0.30 | | −100 | 0.05 | | −130 | 0.00 |

Key EU calculations: - Drill (no survey): EU = 0.25(1) + 0.75(0.05) = 0.25 + 0.038 = **0.288** **0.4** (textbook rounds with different priors) - Sell (no survey): EU = **0.333** (utility of $90K certain)

> **Critical insight:** Under utilities, **Sell without survey** wins. Under monetary EP, **Drill without survey** wins. Utility theory reverses the decision because it penalizes the catastrophic loss outcome far more heavily.

Sensitivity Analysis on Utility

Vary the prior probability of oil (p) to find the **crossover point** — where the decision switches:

- EU(Drill) = p × U(700) + (1−p) × U(−100) = p(1) + (1−p)(0.05) = 0.05 + 0.95p
- EU(Sell) = U(90) = 0.333 (constant, certainty)

Set equal: $0.05 + 0.95p = 0.333 \rightarrow 0.95p = 0.283 \rightarrow \mathbf{p} \quad \mathbf{0.298}$

So: **Sell unless P(oil) $>$ ~30%** (with these utility values). Compare this to the monetary crossover which requires a much higher probability of oil to justify drilling — utility lowers the bar for selling because it places heavy weight on avoiding the loss.

> *Note: Exact crossover values depend on the specific utility function used. The textbook sensitivity analysis gives $p = 0.625$ using a simplified utility model.*

Steps for Practical Decision Analysis

Step	Action	Tools
1	Define alternatives and states of nature	Payoff table
2	Assign prior probabilities	Domain expertise, historical data
3	Consider gathering information	EVPI and EVSI calculations
4	Update probabilities if information gathered	Bayes' theorem
5	Build decision tree	Squares (decisions), circles (events)
6	Evaluate with Bayes' rule (EMV)	Backward induction
7	Assess risk attitude	Equivalent lottery method
8	Construct utility function if needed	Elicitation + interpolation
9	Re-evaluate with expected utility	Backward induction with U(x)
10	Conduct sensitivity analysis	Vary p, vary utility assumptions

---

Common Exam Traps in Decision Analysis

- **Forgetting to use posterior probabilities** after a survey — always update with Bayes' theorem before computing the post-survey decision tree nodes.
- **Confusing EVPI and EVSI** — EVPI assumes perfect information; EVSI adjusts for the survey's actual accuracy. EVSI EVPI always.

- **Assigning utility to payoffs vs. expected values** — utility is assigned to terminal payoffs, not to expected values. Never take U(EP); always compute E[U(payoff)].
- **Sign of deviations in utility interpolation** — always check that your interpolated U(x) is between U(x) and U(x) and follows the concave/convex shape of the curve.
- **Risk attitude changes the decision, not just the calculation** — if a question says the company is "risk-averse," that is the signal to apply utility theory rather than raw EMV.

---

3. MCDA & Goal Programming

What is MCDA?

Multiple Criteria Decision Analysis (MCDA) handles problems with **multiple, often conflicting objectives** that cannot be reduced to a single measure. Unlike standard LP (one objective function), MCDA acknowledges that real decisions involve trade-offs — you cannot simultaneously maximize profit, minimize investment, and hit exact employment targets with one variable.

Key insight: In MCDA there is rarely a single "optimal" solution. The goal is to find the **best compromise** given the decision-maker's priorities.

Where MCDA fits in the decision analysis landscape: | Method | Handles uncertainty? | Multiple objectives? | Use case | |———|———————|———————|———————-| | Decision trees / Utility theory | Yes | Single | Risk and uncertainty under one criterion | | LP | Deterministic | Single | Optimize one objective with constraints | | **Goal Programming** | Deterministic | Multiple | Balance several goals simultaneously | | Simulation + Optimization | Yes | Multiple | Complex stochastic multi-goal systems |

Goal Programming — Core Concept

Goal Programming (GP) is a variant of LP that **minimizes weighted deviations from a set of target goals** rather than optimizing a single objective. The idea: since you can't perfectly achieve all goals at once, minimize the total "damage" from missing them.

Key terminology: - **Goal** — a target value for some objective (not a hard constraint). - **d** — underachievement deviation: how far goal i falls *below* target. - **d** — overachievement deviation: how far goal i goes *above* target. - **Weight (w)** — penalty cost per unit of deviation; reflects relative priority.

Important sign logic: - d and d are always 0. - At any feasible point, at most one of {d , d } can be positive (they cannot both deviate at the same time). - If the goal is exactly met: d = d = 0.

Three Types of Goals

Goal Type	Inequality	Penalize	Do NOT penalize	Example
Lower one-sided	LHS Target	d (missing the floor)	d (exceeding is fine)	Profit $125M
Upper one-sided	LHS Target	d (exceeding the ceiling)	d (falling short is fine)	Investment $55M
Two-sided	LHS = Target	Both d and d	—	Employment = 4,000

Memory rule: Ask yourself — "which direction of deviation is bad?" - If being *below* the target is bad → penalize d . - If being *above* the target is bad → penalize d . - If being *either side* of the target is bad → penalize both.

General Goal Constraint Form

Every goal is written as an **equality constraint** by introducing d and d :

$$f_i(x) + d_i^- - d_i^+ = T_i$$

Where: - f_i(x) = the left-hand side expression for goal i - T_i = target value - d 0, d 0

Objective Function (Non-preemptive):

$$\text{Minimize } Z = \sum_i (w_i^- d_i^- + w_i^+ d_i^+)$$

Only include the deviations that are penalized for each goal type.

Dewright Company — Full Worked Example

Context: Dewright Co. is replacing discontinued products with three new ones (Products 1, 2, 3). Goal is to find the best production mix.

Data:

Goal	Product 1 (x)	Product 2 (x)	Product 3 (x)	Target	Weight
Profit	5	7	8	55	3 per unit d
$(M)\|12\|9\|15\| \geq 125\|5perunitd^{-}\|\|Employment(hundreds)\|5\|3\|4\| = 40\|4perunitd^{-}, 2perunitd^{+}\|\|Investment$(M)					

Step 1 — Write goal constraints:

1. **Profit (125, lower one-sided):**

$$12x_1 + 9x_2 + 15x_3 + d_1^- - d_1^+ = 125$$

   Penalize d only (underachievement of profit is bad; exceeding is welcome).

2. **Employment (= 40, two-sided):**

$$5x_1 + 3x_2 + 4x_3 + d_2^- - d_2^+ = 40$$

   Penalize both d and d (too few or too many employees both matter).

3. **Investment (55, upper one-sided):**

$$5x_1 + 7x_2 + 8x_3 + d_3^- - d_3^+ = 55$$

   Penalize d only (overspending capital is bad; underspending is fine).

Step 2 — Objective function:

$$\text{Minimize } Z = 5d_1^- + 4d_2^- + 2d_2^+ + 3d_3^+$$

Step 3 — Non-negativity:

$$x_1, x_2, x_3 \geq 0 \quad \text{and} \quad d_i^-, d_i^+ \geq 0 \text{ for } i = 1, 2, 3$$

Step 4 — Evaluate candidate mixes:

Mix 1 (textbook solution): - Profit = \$116.25M → d = 8.75, d = 0 - Employment = 4,000 → d = 0, d = 0 - Investment = \$55M → d = 0, d = 0 - **Penalty Z = 5(8.75) + 0 + 0 + 0 = 43.75** ← lower is better

Mix 2: - **Penalty = 75** (higher — worse solution)

Conclusion: Mix 1 is better. Sometimes you cannot achieve all goals — the GP framework explicitly surfaces this trade-off and gives a defensible, weighted compromise.

Non-Preemptive vs. Preemptive Goal Programming

Non-preemptive (Weighted GP): - All goals are roughly comparable in importance. - A single objective function with weights captures relative priority. - **All goals are optimized simultaneously.** - Suitable when trade-offs between goals are acceptable.

Preemptive (Lexicographic GP): - Goals are ranked in a strict **hierarchy of priority levels** (P1 > P2 > P3 …). - **P1 goals are fully optimized first**, then P2 is optimized without degrading P1, etc. - Suitable when some goals must not be sacrificed for others under any circumstances. - Example: "Maintaining 4,000 jobs is Priority 1; profit is Priority 2" — never trade employment for profit.

How to identify which type to use: - If the problem says "goals are equally important" or gives weights → non-preemptive. - If the problem says "first priority is X, second priority is Y" → preemptive.

Formulation Checklist

Step	What to do
1	Identify decision variables (what you control)
2	For each goal: write f(x) = Target as equality using d and d
3	Identify goal type (lower/upper/two-sided) and which deviation to penalize
4	Assign penalty weights (higher weight = higher priority)
5	Write objective: Minimize Z = Σ(weight × penalized deviation)
6	Add non-negativity for all x , d , d
7	Solve (LP solver, Python, or manual for small problems)
8	Interpret: which goals were achieved, which were missed, and by how much

---

Why Goal Programming Over Standard LP?

In standard LP: Goals become hard constraints — if you can't satisfy one, the problem is infeasible.

In GP: Goals become soft targets — infeasibility is replaced by a penalty for deviation. The model always has a feasible solution (it just measures how far off you are).

Real-world value: Executives rarely have perfectly compatible targets. GP gives a mathematically rigorous way to find the best compromise when perfection is impossible.

Exam Tips for Goal Programming

- **Never penalize the "good" direction of deviation.** Profit above target is never bad; investment below budget is never bad. Only penalize the deviation direction that hurts.
- **Two-sided goals always have both d and d in the objective.** Even if they have different weights.
- **The constraint always has both d and d** — this is the general form. But only the problematic deviation appears in the objective.
- **Check your penalty arithmetic carefully** — it is where exam points are lost. Substitute the solution into each goal constraint to find the deviation magnitudes, then multiply by weights.
- **Higher weight goal is achieved** — a higher weight just means missing that goal is more costly. The model still balances all goals jointly.

---

4. Markov Decision Processes

Key Concepts

- **MDP** = Controlled Markov chain for decision-making under uncertainty.
- **Markovian property:** Future depends only on **current state and decision** — not history.
- **Goal:** Minimize long-run expected average cost per unit time.

MDP Components

Component	Description	Example
States	Possible conditions (i = 0, 1, …, M)	Machine: Good/Minor/Major/Inoperable
Decisions	Actions available in each state (k = 1, …, K)	Do nothing / Overhaul / Replace
C	Immediate cost of decision k in state i	Replace in state 3: $6,000
p (k)	Probability of transitioning i → j under decision k	Transition matrix
Policy R	Set of decisions for each state {d , d , …, d }	R : (1,1,1,3)

Prototype Example — Machine Maintenance

States: 0 (Good), 1 (Minor deterioration), 2 (Major deterioration), 3 (Inoperable)

Base Policy R : Replace only when inoperable (1, 1, 1, 3)

Steady-State Equations (for R): - = (replacement puts machine back to state 0) - = (7/8) + (3/4) - = (1/16) + (1/8) + (1/2) - = (1/16) + (1/8) + (1/2) - + + + = 1

Solution: = 2/13, = 7/13, = 2/13, = 2/13

Expected Average Cost:

$$E(C) = \sum_i C_{i,d_i} \cdot \pi_i = 0(\tfrac{2}{13}) + 1000(\tfrac{7}{13}) + 3000(\tfrac{2}{13}) + 6000(\tfrac{2}{13}) = \frac{25000}{13} \approx \$1,923/\text{week}$$

Comparing Policies by Exhaustive Enumeration

Policy	Decisions (States 0,1,2,3)	E(C)/week
R	Do nothing, Do nothing, Do nothing, Replace	$1,923
R	Do nothing, Do nothing, **Overhaul**, Replace	**$1,667 ← Optimal**
R	Do nothing, Do nothing, Replace, Replace	$1,750
R_d	Do nothing, Replace, Replace, Replace	$2,100

LP Formulation for MDPs (Section 19.3)

Why LP? Exhaustive enumeration is infeasible for large problems (10 states × 5 decisions = 5^1 policies).

Variable: y = steady-state probability of being in state i and choosing action k.

Objective:

$$\text{Minimize } Z = \sum_i \sum_k C_{ik} \cdot y_{ik}$$

Constraints: 1. **Normalization:** Σ Σ y = 1 2. **Transition balance** (for each state j, inflow = outflow):

$$\sum_k y_{jk} = \sum_i \sum_k p_{ij}(k) \cdot y_{ik}$$

3. **Non-negativity:** y 0

Example (Dewright Prototype): - Variables: y , y , y , y , y , y , y - State 0 balance: y = y + y + y (only reached via replacement)

LP solution directly identifies optimal policy — no need to enumerate all policies.

Recovering the Policy from LP Solution

After solving LP, for each state i:

$$d_i^*(R) = \arg\max_k y_{ik} \quad \text{(the action with positive } y_{ik})$$

---

5. Queueing Theory

System Components

Component	Description
Arrival pattern	How customers arrive; typically Poisson with rate
Service mechanism	Number of servers s; service rate (exponential)
Queue capacity	Infinite (∞) or finite (N)
Service discipline	FCFS (default), LCFS, SIRO, Priority

Customer behaviors: - **Balking** — leaves without joining (queue too long) - **Reneging** — joins then leaves (impatient) - **Jockeying** — switches between queues

---

Traffic Intensity (Utilization)

$$\rho = \frac{\lambda}{\mu} \quad \text{(single server)} \qquad \rho = \frac{\lambda}{s\mu} \quad \text{(multi-server)}$$

- $<$ **1:** Stable system (queue won't grow forever)
- **1:** Unstable (queue grows indefinitely)

Poisson vs. Exponential Distributions

Feature	Poisson	Exponential
Models	Number of arrivals in fixed time	Time between arrivals
Type	Discrete	Continuous
Formula	P(X=k) = e /k!	f(t) = e
Mean		1/

They are paired: If arrivals are Poisson( ), then inter-arrival times are Exponential( ).

Model I: M/M/1 (∞/FIFO) — Single Server, Infinite Capacity

Key Formulas (memorize W , derive the rest):

$$W_s = \frac{1}{\mu - \lambda} \quad \text{(avg time in system)}$$

$$W_q = \frac{\lambda}{\mu(\mu - \lambda)} \quad \text{(avg wait in queue)} = W_s - \frac{1}{\mu}$$

$$L_s = \frac{\lambda}{\mu - \lambda} = \lambda W_s \quad \text{(avg customers in system)}$$

$$L_q = \frac{\lambda^2}{\mu(\mu - \lambda)} = \lambda W_q \quad \text{(avg customers in queue)}$$

$$P_n = \rho^n(1 - \rho) \quad \text{(probability of n customers)}$$

$$P_0 = 1 - \rho \quad \text{(probability system is empty)}$$

Memory trick: Remember W = 1/(−). Then: - Wq = (/) × W - L = × W - Lq = × Wq

---

Worked Example (M/M/1)

Given: = 30/hr, service time = 90 sec → = 40/hr, = 0.75

Metric	Calculation	Result
L	/(1−) = 0.75/0.25	**3 customers**
Lq	^{2}/(1−) = 0.5625/0.25	**2.25 customers**
Wq	Lq/ = 2.25/30	**4.5 min**
W	Wq + 1/ = 0.075 + 0.025 hr	**6 min**

Model II: M/M/1 (N/FIFO) — Single Server, Finite Capacity

Key difference: Customers are turned away when system is full (N customers).

$$P_0 = \frac{1-\rho}{1-\rho^{N+1}}$$

$$P_N = \rho^N \cdot P_0$$

Effective arrival rate: _eff = (1 − P)

Expected number in system:

$$L = \frac{\rho}{1-\rho} - \frac{(N+1)\rho^{N+1}}{1-\rho^{N+1}}$$

Model III: M/M/s (∞/FIFO) — Multi-Server, Infinite Capacity

- s = number of servers
- Each server has rate ; total capacity = s
- System stable if < s (i.e., = /(s) < 1)
- Formulas are more complex — refer to formula sheet.

Queueing Notation Summary: M/M/s/N

Symbol	Meaning
M (first)	Markovian (Poisson) arrivals
M (second)	Markovian (Exponential) service
s	Number of servers
N	System capacity (∞ if omitted)
FIFO	Queue discipline

---

6. Simulation I

What is Simulation?

Simulation **mimics** a real system using a computer to model stochastic (random) behavior. Used when: - Systems are **too complex** for analytical solutions. - Randomness is **essential** to model realistically. - **"What-if" testing** without real-world costs.

Not guessing — it is a structured statistical experiment.

When to Use Simulation vs. Analytical Methods

Use Analytical (e.g., LP, Queueing)	Use Simulation
Problem is mathematically tractable	System too complex
Exact solution possible	Randomness is essential
Quick sensitivity analysis needed	Real-world detail matters
	Comparing policies before implementation

Key Components of a Simulation Model

Component	Role
System State	Variables describing the system (e.g., # in queue)
Events	Actions that change state (arrivals, departures)
Clock	Tracks simulated time
Randomness	Generates stochastic events via random numbers
Rules	Logic for state transitions
Performance Metrics	Outputs (avg wait, utilization, etc.)

Types of Simulation

- **Discrete-Event Simulation (DES):** State changes at specific time points (e.g., customer arrivals). Most common in OR.
- **Continuous Simulation:** State changes smoothly over time (e.g., fluid flow, aircraft).

Random Number Generation

Pseudo-Random Numbers

- Generated by **algorithms**, not truly random.
- **Deterministic** if seed is known (reproducible).
- Appear random and statistically acceptable.

Mixed Congruential Method (MCM)

$$x_{n+1} = (a \cdot x_n + c) \mod m$$

- **a** = multiplier, **c** = increment, **m** = modulus, **x** = seed
- **Cycle length:** How many numbers before repetition. Maximum = m.
- Good generators have **long cycle lengths**.

Example: a=5, c=7, m=8, x =4 - x = (5×4 + 7) mod 8 = 27 mod 8 = **3** - x = (5×3 + 7) mod 8 = 22 mod 8 = **6** - Sequence: 3, 6, 5, 0, 7, 2, 1, 4 (cycle length = 8)

Converting to Uniform Random Numbers [0,1]:

$$U = \frac{x_n + 0.5}{m}$$

Simulation Procedure (Step-by-Step)

1. **Set up probability distribution** for the random variable.
2. **Build cumulative probability distribution.**
3. **Assign random number intervals** proportional to probabilities.
4. **Draw random numbers** and match to intervals → get values.
5. **Run simulation** for required periods.
6. **Record and analyze results.**

Dentist Clinic Example

Setup: 8 patients at 30-min intervals (8AM–12PM). Random numbers: 40, 82, 11, 34, 25, 66, 17, 79.

Category	Time	Prob	Cumulative	RN Range
Filling	45 min	0.40	0.40	01–40
Crown	60 min	0.15	0.55	41–55

Category	Time	Prob	Cumulative	RN Range
Cleaning	15 min	0.15	0.70	56–70
Extraction	45 min	0.10	0.80	71–80
Check-up	15 min	0.20	1.00	81–100

Results: - Total waiting time = 180 minutes - **Average waiting time = 22.5 minutes** - **Doctor idle time = 0 minutes**

---

7. Simulation II

Pseudo-Random vs. Uniform Random Numbers

Feature	Pseudo-Random	Uniform Random
Format	Integer (e.g., 3, 7)	Real number (e.g., 0.333)
Range	0 to m−1	0.0 to 1.0
Generated by	MCM algorithm	Scaling pseudo-randoms
Nature	Deterministic (with seed)	Appears continuous [0,1]

Inverse Transformation Method

Core idea: Any distribution can be generated from uniform random numbers U[0,1] using its inverse CDF.

$$X = F^{-1}(U)$$

Steps: 1. Generate U ~ Uniform[0,1]. 2. Set F(x) = U. 3. Solve for x = F ^{1}(U) → this is your random observation.

Visual intuition: Pick U on the y-axis of the CDF curve, project horizontally to the curve, then down to the x-axis to get X.

Distributions — Inverse Transform Formulas

Uniform Distribution U(a, b):

$$x = a + (b - a) \cdot r$$

Where r is a uniform random number.

Example: a=2, b=5, r=0.4 → x = 2 + 3(0.4) = **3.2**

Exponential Distribution f(x) = e :

$$x = -\frac{\ln(r)}{\lambda}$$

Excel: `= -LN(RAND())/`

Erlang Distribution (sum of k exponentials):

$$x = -\frac{1}{k\lambda}\ln(r_1 \cdot r_2 \cdots r_k)$$

Generate k uniform numbers and multiply inside the log.

Normal Distribution N(,):

- **CLT approach:** Sum 12 uniform numbers, subtract 6 → N(0,1). Then: X = + (Σr − 6)
- **Exact methods:** Box-Muller transform; or in Excel: `=NORM.INV(RAND(), , )`

Excel Simulation Formulas

Distribution	Excel Formula	Output
Uniform [0,1]	`=RAND()`	Flat, 0 to 1
Exponential (rate=)	`=-LN(RAND())/`	Right-skewed
Normal (,)	`=NORM.INV(RAND(), , )`	Bell curve
Erlang (k,)	`=GAMMA.INV(RAND(), k, 1/ )`	Right-skewed

Bank Queue Simulation Example

Parameters: = 4 customers/hr, = 5 customers/hr (Erlang k=2), 8-hour simulation.

Column	Formula
Interarrival time	`=-LN(RAND())/4`
Arrival time	Previous arrival + interarrival
Service time	`=GAMMA.INV(RAND(), 2, 1/5)`
Start time	`=MAX(prev_end, arrival)`
End time	Start + Service
Waiting time	Start − Arrival

Analyze: Average waiting time, max waiting time, server utilization = Σservice time / total time.

Simulation Workflow (Complete Process)

1. Collect data → fit probability distributions.
2. Generate pseudo-random integers (MCM).
3. Convert to U(0,1) uniform numbers.
4. Transform to target distributions (inverse transform).
5. Build simulation model (logic, events, rules).
6. Run experiments (warm-up period + replications).
7. Analyze output (statistics, confidence intervals).
8. Validate and report results.

Simulation Optimization

Problem: Simulation alone gives statistical estimates — it doesn't inherently optimize.

Simulation Optimization combines simulation with optimization to find the best system configuration.

Approaches by State Space Size:

State Space	Method
Small (few configs)	Ranking & Selection (Bechhofer's Procedure) — simulate N runs per option, compare averages
Large	Fully Sequential Procedures (Paulson/KN) — eliminate poor options early, focus sampling on promising ones
Continuous	Gradient-based or random search methods

Fully Sequential advantage: Reduces unnecessary sampling by discarding inferior options early — useful when simulation runs are expensive.

Outline of a Major Simulation Study

Step	Key Actions
1. Formulate Problem	Define objectives, constraints, performance measures with stakeholders
2. Collect Data & Build Model	Fit distributions, define events, flow diagrams, state variables
3. Validate Model	Expert walk-through; compare outputs to real data
4. Select Software	Excel (simple), Python/R (flexible), dedicated simulators
5. Test Validity	Field tests, sensitivity analysis, animation checks
6. Plan Simulations	Choose run length and number of replications
7. Run & Analyze	Execute, record metrics, compute confidence intervals
8. Report	Written report: methodology, validation, results, recommendations

Best practices: Involve stakeholders early; use variance-reduction techniques; document all assumptions.

Final Note: Decision Analysis — The Complete Domain Map

Decision Analysis spans three chapters in this course (Ch. 2 Utility Theory, Ch. 3 MCDA/GP, and overlaps with Ch. 4 MDP). Here is a unified map of the entire decision analysis domain to anchor your understanding before the exam.

The Decision Analysis Landscape

The field answers one core question: **"Given uncertainty and competing objectives, what should I do?"** Different tools handle different combinations of uncertainty and complexity:

```
                    Single Objective          Multiple Objectives

No Uncertainty      Linear Programming        Goal Programming
(Deterministic)     (one criterion LP)        (MCDA / Dewright)

Uncertainty         Decision Trees            Multi-attribute
(Stochastic)        + Utility Theory          Utility Theory
                    + Bayes' Theorem          (advanced MCDA)
```

```
Sequential          Markov Decision          Multi-objective MDP
Decisions over      Processes (MDP)          (advanced)
Time
```

Where each chapter falls: - **Payoff tables + Bayes' rule** → Stochastic, single objective, one-shot decision. - **Utility theory** → Stochastic, single objective, one-shot, with risk attitude. - **Decision trees + Bayesian updating** → Stochastic, single objective, multi-stage. - **Goal Programming** → Deterministic, multiple objectives. - **MDP** → Stochastic, single objective (minimize cost), sequential over time.

Decision Criteria Summary — When to Use What

Criterion	Probabilities needed?	Risk attitude	When to use
Maximin	No	Extreme risk-aversion	Unknown probabilities; pessimistic decision-maker
Maximax	No	Risk-seeking	Unknown probabilities; optimistic decision-maker
Minimax Regret	No	Regret-minimizing	Decision-maker focuses on avoiding post-decision regret
Equally Likely	No (assumes equal)	Risk-neutral	Complete ignorance about probabilities
Bayes' (EMV)	Yes	Risk-neutral	Probabilities known; decision-maker is risk-neutral
Expected Utility	Yes	Risk-averse or risk-seeking	Probabilities known; risk attitude matters

---

Information Value — Decision Sequence

```
Step 1: Make decision with PRIOR probabilities only (no survey)
        ↓ Bayes' Decision Rule gives: EP(best alternative without info)

Step 2: Calculate EVPI to bound the value of any information
        EVPI = EP(with perfect info) − EP(best without info)
        → If cost of survey > EVPI, don't bother with any survey

Step 3: If survey is available, use Bayes' theorem to get POSTERIOR probabilities
        P(S  | finding) = [P(finding | S ) × P(S )] / P(finding)

Step 4: Build the full decision tree WITH survey option
        → Compute EVSI = EP(with survey) − EP(without survey)
        → If survey cost < EVSI, the survey is worth taking

Step 5: Compare all strategies, choose the optimal policy
```

Bayesian Updating — Complete Procedure

Given: - Prior probabilities P(S), P(S), ..., P(S) for each state of nature. - Conditional likelihoods P(F | S) for each possible finding F given each state.

Compute: 1. **Joint probabilities:** P(S F) = P(F | S) × P(S) for each state. 2. **Marginal probability of finding:** P(F) = Σ P(S F). 3. **Posterior probabilities:** P(S | F) = P(S F) / P(F).

Tabular format (always use this on the exam — it reduces errors):

State S	P(S)	P(F|S)	P(S F) = col2×col3	P(S |F) = col4/ΣP(F)
S	0.25	0.6	0.150	0.150/0.250 = 0.60
S	0.75	0.2	0.100	0.100/0.250 = 0.40
Sum	**1.00**		**P(F) = 0.250**	**1.00**

Repeat the entire table for each possible finding (e.g., favorable AND unfavorable survey results produce two separate posterior distributions).

Utility Theory — The Full Argument Chain

Understanding *why* utility theory is needed is as important as knowing *how* to apply it. Here is the complete logic chain:

Problem with EMV: Two decision-makers facing the same EMV-optimal choice may legitimately make different decisions if they have different risk tolerances. EMV treats $1M gain and $1M loss as perfect opposites — but for most people they are not (loss looms larger psychologically and financially).

The utility function solves this by mapping money → subjective value: - It is **personal** — each decision-maker has their own utility function. - It is **elicited through choices**, not calculated mathematically. - It **captures diminishing returns** to wealth (for risk-averse individuals). - Once constructed, EU replaces EP in all decision tree calculations.

The equivalent lottery method in plain language: > You are offered Choice A: receive $M for certain. OR Choice B: enter a lottery with probability p of winning the maximum payoff and (1−p) of getting the worst payoff. At what value of p would you be completely indifferent between A and B? That probability p is your utility for $M.

Key properties of any valid utility function: - U(worst) = 0 (by construction). - U(best) = 1 (by construction). - All intermediate values: 0 U(x) 1. - Monotonically increasing: more money → higher utility (for a rational decision-maker). - Shape encodes risk attitude: concave = risk-averse, convex = risk-seeking.

Comparing EMV vs. EU Decisions — Goferbroke Summary

Decision	EMV Approach	Utility Approach
Drill (no survey)	EP = $100K ← **Chosen**	EU = 0.288
Sell (no survey)	EP = $90K	EU = 0.333 ← **Chosen**
Reason for difference	Ignores risk attitude	Penalizes catastrophic −$100K loss
Key lesson	EMV is right for risk-neutral	EU is right when stakes are asymmetric

The $10K difference in EMV is small. But the utility function reveals that the owner dreads the −$100K outcome so strongly (U = 0.05, near zero) that even a certain $90K (U = 0.333) is far preferable. **The utility function makes this asymmetry explicit and quantifiable.**

Goal Programming — The Decision Analysis Connection

Goal Programming bridges decision analysis and LP optimization:

Feature	Standard Decision Analysis	Goal Programming
Nature of alternatives	Discrete (drill/sell)	Continuous (how much of x , x , x)
Uncertainty	Yes (states of nature)	No (deterministic goals)
Number of objectives	One (EMV or EU)	Multiple (profit, employment, investment)
How trade-offs are handled	Utility function weights outcomes	Penalty weights weight deviations
Solution concept	Optimal decision	Best compromise

The unifying thread: Both utility theory and goal programming ask the decision-maker to **make their priorities explicit through numbers** — utility weights in one case, penalty weights in the other. Both convert subjective preferences into a solvable mathematical model.

Decision Analysis in the Real World — Application Domains

Finance and Investment: - Portfolio selection under uncertainty (maximize return, minimize risk). - Whether to hedge currency exposure (EMV of hedging vs. not). - Valuing real options (e.g., option to expand a project if demand is high).

Healthcare: - Treatment selection: surgery vs. chemotherapy vs. watchful waiting, with different outcomes and probabilities. - Public health policy: e.g., vaccination programs analyzed via large decision trees. - Resource allocation across departments with multiple competing objectives (MCDA).

Business Strategy: - Product launch decisions (invest in R&D with uncertain market success). - Make-or-buy decisions under cost and quality uncertainty. - Market entry decisions in new geographies with political and economic risk.

Government and Public Policy: - Infrastructure investment (build a dam vs. flood insurance vs. relocation) balancing cost, safety, and environmental impact — classic multi-criteria problem. - Disaster response resource allocation — GP balances coverage, speed, and budget.

Engineering and Operations: - Machine maintenance policies (see also: MDP in Ch. 4). - Quality control: decision trees determine inspection policies under defect probability uncertainty. - Supply chain design: trade-off between cost, lead time, and resilience (MCDA).

The Goferbroke case as a template: Nearly every real decision analysis problem follows the same structure — define alternatives, model uncertainty with probabilities, incorporate the decision-maker's risk attitude, value information, and conduct sensitivity analysis. Master this template and you can analyze any decision problem.

Topic A	Topic B	Connection
Game Theory	LP	LP solves mixed strategies; dual = Player 2's problem
MDP	Markov Chains	MDP extends Markov chains by adding decisions
MDP	LP	LP formulation finds optimal policy more efficiently than enumeration
Queueing	Simulation	Simulation can model queues too complex for M/M/s formulas
Simulation	Decision Analysis	Simulation generates the distributions; decision analysis uses them
Goal Programming	LP	GP is LP with deviational variables replacing hard constraints

---

Formula Quick-Reference Sheet

Game Theory

- **Saddle point:** row min = column max
- **Mixed strategy EP:** Σ p · x · y
- **Player 1 LP:** Maximize v s.t. Σ p x v j, Σx =1

Queueing (M/M/1)

- = /
- W = 1/(−)
- Wq = /[(−)]
- L = W = /(−)
- Lq = Wq = ²/[(−)]
- P = 1− , P = (1−)

Simulation — Inverse Transform

- Uniform(a,b): x = a + (b−a)r

- Exponential( ): x = −ln(r)/
- Normal: Excel `=NORM.INV(RAND(), , )`

MDP

- E(C) = Σ C ,d(R) ·
- Steady state: Solve = Σ p and Σ =1

Goal Programming

- Goal constraint: (function) + d − d = Target
- Minimize Z = Σ w d + w d

Utility Theory

- Lottery method: U(M) = p (when indifferent between M and [p×max, (1-p)×min])
- Interpolation: U(x) = U(x) + [(x−x)/(x −x)] × [U(x)−U(x)]

Good luck on the exam!